Filière :	Développement des Systèmes d'Information	Durée :	2 Heures
Épreuve :	Développement des Applications Informatiques DAI	Coefficient :	50

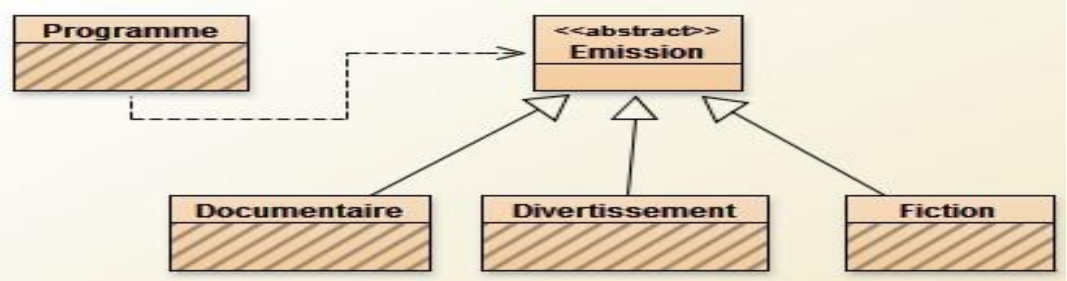
PARTIE : PROGRAMMATION RESEAU (CLIENT/SERVEUR)

1. Programmer un serveur classe **Serveur** qui accepte la connexion d'un client, affiche le message « Connexion établie» et envoi le message « Bonjour mon client DSI2 »
2. Programmer la classe **Client** qui demande la connexion au serveur et affiche « Demande de connexion », une fois le serveur accepte la demande, le client envoi le message « Merci serveur… » vers le serveur .

PARTIE : POO (JAVA)

Service de programmation des émissions

Le modèle abstrait des données est la suivant :



1) Le détail de la classe Emission est le suivant :

```
public abstract class Emission implements
Comparable{     private String TitreEm;  private
String duréeEm;
private Date dateEm;
public Emission(String TitreEm, String duréeEm, Date dateEm) { }
public Emission(String TitreEm, String duréeEm) { }
public  int compareTo(Object ob) { }
@Override
public String toString() { }
public abstract String getInfos(); }
```

a- Définir les deux constructeurs pour initialiser les attributs de cette classe, le constructeur avec deux arguments initialise la date d'émission par celle du système.

b- Définir la méthode **compareTo** qui compare deux émissions selon leurs titres insensible à la casse.

c- Donner la définition de la méthode toString, afin de retourner une chaine porteuse d'informations sur une tache, la chaine aura la forme suivante : **Titre : xxxx , la durée : xxxxx , Date d'émission : jour/mois/année.**

2) Le détail de la classe Documentaire le suivant :

```
public class Documentaire extends Emission{
private String theme;
   public Documentatire( ) { }
   public  String getInfos( ) { }
```

a- Proposer un constructeur adéquat permettant d'initialiser tous les attributs b- Donner le code de la méthode **getInfos( )** permettant de retourner une chaine sous la forme suivante :

Titre : xxxx , la durée : xxxxx , Date d'émission : jour/mois/année. Thème : xxxxxxxx

3) Le détail de la classe Fiction le suivant :

```
public class Fiction extends Emission{ private
String réalisateur, producteur;
   public Fiction( ) {  }     public  String getInfos(){ }
}
```

a- Proposer un constructeur adéquat permettant d'initialiser tous les attributs b- Donner le code de la méthode **getInfos( )** permettant de retourner une chaine sous la forme suivante :

Titre : xxxx , la durée : xxxxx , Date d'émission : jour/mois/année.
Réalisateur : xxxxxxxx
Producteur : xxxxxxxx

4) Le détail de la classe Programme le suivant :

```
public class Programme{
private ArrayList<Emission> Liste=new ArrayList<Emission>() ;
public boolean addEmission(Emission x) { }
public  Emission delEmission(int p){ }
public  boolean delEmission(Emission p){ }
public  Emission setEmission(int p,Emission x){ }
public  String toString( ){ }
public boolean saveAll(String f) { }
public boolean loadAll(String f) { }
}
```

Donner le code des méthodes suivantes :

a- addEmission(..) : permet d'ajouter à la collection une nouvelle émission et retourne l'état de l'opération.

b- delEmission(int) : permet de supprimer une émission de la collection en se basant sur un index et retourne l'émission qui vient d'être supprimée, si l'index est invalide, on doit lancer une exception personnalisée que l'on nomme **ErrIndexException** (cette classe doit être créée).

c- delEmission(Emission ..) : permet de supprimer une émission de la collection en se basant sur un objet Emission et retourne l'état de suppression.

d- setEmission(…..) : permet de modifier une émission déjà existante, si l'index est invalide on lance l'exception : **ErrIndexException**

e- toString( ) : retourne une chaine contient toutes les émissions **f- saveAll** et **loadAll** permettent respectivement de sauvegarder toutes les émissions de la collection dans un fichier d'objet et de charger toutes les émissions depuis un fichier d'objet dans notre collection.